SkillForge – Database Design

SQLite-compatible relational database design based on the locked conceptual ERD

Scope: This design implements only the approved SkillForge player, challenge, inventory, skill/ability, settings, progression and reward-state data. It does not add admin, multiplayer, currency/economy, cloud, or other unsupported features.

PLAYER

Column	SQLite Type	Key / Constraint	Purpose
Player_ID	TEXT	PK, NOT NULL	Player identity
Username	TEXT	NOT NULL, UNIQUE	Player username
Level	INTEGER	NOT NULL, DEFAULT 1	Player level
XP	INTEGER	NOT NULL, DEFAULT 0	Experience points
Play_Time	INTEGER	NOT NULL, DEFAULT 0	Accumulated play time (stored as an integer value)

CHALLENGE

Column	SQLite Type	Key / Constraint	Purpose
Challenge_ID	TEXT	PK, NOT NULL	Challenge identifier
Challenge_Name	TEXT	NOT NULL	Challenge name
Challenge_Level	INTEGER	NOT NULL	Challenge level; exact values are defined during implementation

CHALLENGE_PROGRESS

Column	SQLite Type	Key / Constraint	Purpose
Player_ID	TEXT	PK, FK, NOT NULL	References PLAYER
Challenge_ID	TEXT	PK, FK, NOT NULL	References CHALLENGE
Completion_Status	INTEGER	NOT NULL, DEFAULT 0, CHECK 0/1	Challenge completion state
Reward_Claimed	INTEGER	NOT NULL, DEFAULT 0, CHECK 0/1	Reward-claimed state after valid completion

SKILL_ABILITY

Column	SQLite Type	Key / Constraint	Purpose
Skill_ID	TEXT	PK, NOT NULL	Skill/ability identifier

Skill_Name	TEXT	NOT NULL	Skill/ability name

PLAYER_SKILL

Column	SQLite Type	Key / Constraint	Purpose
Player_ID	TEXT	PK, FK, NOT NULL	References PLAYER
Skill_ID	TEXT	PK, FK, NOT NULL	References SKILL_ABILITY
Unlock_Status	INTEGER	NOT NULL, DEFAULT 0, CHECK 0/1	Whether the player has unlocked the skill/ability

INVENTORY

Column	SQLite Type	Key / Constraint	Purpose
Inventory_ID	INTEGER	PK, AUTOINCREMENT	Inventory record identifier
Player_ID	TEXT	FK, NOT NULL	References PLAYER
Item_Name	TEXT	NOT NULL	Inventory item name
Quantity	INTEGER	NOT NULL, DEFAULT 1, CHECK >= 0	Quantity of the item

SETTINGS

Column	SQLite Type	Key / Constraint	Purpose
Player_ID	TEXT	PK, FK, NOT NULL	References PLAYER; enforces one settings record per player
Interface_Settings	TEXT	—	Current interface-setting state
Audio_Settings	TEXT	—	Current audio-setting state

Relationships and Implementation Rules

ERD relationship	Database implementation	Cardinality
PLAYER — HAS — SETTINGS	SETTINGS.Player_ID → PLAYER.Player_ID; PK + FK	1 : 1
PLAYER — OWNS — INVENTORY	INVENTORY.Player_ID → PLAYER.Player_ID	1 : M
PLAYER — COMPLETES — CHALLENGE	CHALLENGE_PROGRESS resolves M:M	M : M
PLAYER — UNLOCKS — SKILL_ABILITY	PLAYER_SKILL resolves M:M	M : M

1. SQLite does not have a separate BOOLEAN storage class, so the status flags use INTEGER values 0/1 with CHECK constraints.

2. CHALLENGE_PROGRESS stores the two attributes attached to the ERD COMPLETES relationship: Completion_Status and Reward_Claimed.

3. PLAYER_SKILL is the junction table required to implement the M:M PLAYER–SKILL_ABILITY relationship.

4. CHALLENGE_PROGRESS is the junction table required to implement the M:M PLAYER–CHALLENGE relationship.

5. SETTINGS uses Player_ID as both PK and FK so one player has one settings record.

6. Exact challenge names, challenge levels, skill names and inventory item values are not invented here; they can be populated during implementation.

7. Interface_Settings and Audio_Settings are kept as simple state fields. No graphics/gameplay settings are added.

8. SaveManager remains the sole persistent save/load handler at the application architecture level; these tables define the local SQLite persistence structure.

Final table set

PLAYER, CHALLENGE, CHALLENGE_PROGRESS, SKILL_ABILITY, PLAYER_SKILL, INVENTORY, SETTINGS.